CS-471: Machine Learning

Week 6 — Decision Tree Split Criteria, Stopping Conditions, ID3, C4.5 and CART Algorithms

Instructor: Dr. Hashir Kiani Sources: Lectures 6-1 & 6-2

Comprehensive Study Notes

Week 6

Contents

1 The Core Question: Which Attribute to Split On? 2

2 The Golf Dataset (Reference) 2

3 Gini Impurity 2
- 3.1 Gini Impurity of a Set . . . 2
- 3.2 Gini Impurity of a Split (Test) . . . 3

4 Entropy and Information Gain 4
- 4.1 Entropy of a Set . . . 4
- 4.2 Entropy After a Split . . . 4
- 4.3 Information Gain . . . 4
- 4.4 Entropy vs. Gini — Which to Use? . . . 6

5 Decision Tree Stopping Criteria 6
- 5.1 List of Standard Stopping Criteria . . . 6
- 5.2 Edge Cases and How to Handle Them . . . 7

6 The ID3 Algorithm 8
- 6.1 Overview . . . 8
- 6.2 ID3 Algorithm — Step by Step . . . 8
- 6.3 Advantages of ID3 . . . 9
- 6.4 Limitations of ID3 . . . 9

7 The C4.5 Algorithm 9
- 7.1 Overview . . . 9
- 7.2 Gain Ratio (Replacing Information Gain) . . . 10
- 7.3 Handling Continuous Attributes . . . 11
- 7.4 Handling Missing Values . . . 11
- 7.5 Pruning in C4.5 . . . 12
- 7.6 Advantages and Limitations of C4.5 . . . 12

8 Comparison: ID3 vs. C4.5 **13**

9 The CART Algorithm **13**
9.1 Overview 13
9.2 How CART Works 13
9.3 Splitting Criteria 14
9.4 CART Stopping Criteria 14
9.5 CART for Regression — Worked Example 14

10 Summary: ID3 vs. C4.5 vs. CART **15**

11 Practice Questions **15**

12 Key Takeaways — Week 6 **17**

1 The Core Question: Which Attribute to Split On?

The fundamental challenge in building a decision tree is: at each node, which feature should we test? We want to choose the feature whose split best separates the data into *pure* subsets (subsets where all examples belong to the same class).

Two standard mathematical measures of "impurity" are used:

1. **Gini Impurity** — used by the CART algorithm
2. **Entropy** — used by ID3 and C4.5

Both measures equal 0 for a pure set (all one class) and are maximised when classes are equally mixed.

2 The Golf Dataset (Reference)

All worked examples in this week's lectures use the following dataset ($N = 12$ training examples, $K = 2$ classes: Yes/No).

Ex.	Outlook	Temperature	Humidity	Windy	Play Golf?
1	Rainy	Hot	High	False	**No**
2	Rainy	Hot	High	True	**No**
3	Overcast	Hot	High	False	**Yes**
4	Sunny	Mild	High	False	**Yes**
5	Sunny	Cool	Normal	False	**Yes**
6	Sunny	Cool	Normal	True	**No**
7	Overcast	Cool	Normal	True	**Yes**
8	Rainy	Mild	High	False	**No**
9	Sunny	Mild	Normal	False	**Yes**
10	Rainy	Mild	Normal	True	**Yes**
11	Overcast	Mild	High	True	**Yes**
12	Sunny	Mild	High	True	**No**

Class counts in full dataset: Yes = 7, No = 5, Total = 12.

3 Gini Impurity

3.1 Gini Impurity of a Set

$$\boxed{\text{Gini}(S) = 1 - \sum_{k=1}^{K} p_k^2}$$

where p_k is the proportion of class k in set S, and K is the number of classes.

Properties:

- Gini = 0: set is *pure* (all examples same class).

- Gini = 0.5 (for binary classification): maximum impurity (50%/50% split). In general: $\text{Gini}_{\max} = 1 - 1/K$.
- Gini ranges from 0 to $(1 - 1/K)$.

Real-Life Analogy: Think of a bag of coloured marbles. If all marbles are the same colour (pure), the probability of picking two of the same colour at random is 1 — Gini = 0. If half are red and half blue, the bag is maximally "impure" — Gini = 0.5.

Example

Gini Impurity of the full golf dataset:
$p_{\text{Yes}} = 7/12$, $p_{\text{No}} = 5/12$.

$$\text{Gini}(S) = 1 - \left(\frac{7}{12}\right)^2 - \left(\frac{5}{12}\right)^2 = 1 - \frac{49}{144} - \frac{25}{144} = 1 - \frac{74}{144} = \frac{70}{144} \approx \mathbf{0.486}$$

3.2 Gini Impurity of a Split (Test)

When we split on feature A, the resulting weighted average Gini impurity is:

$$\boxed{\text{Gini}(A) = \sum_{\text{values } v \text{ of } A} \frac{|S_v|}{|S|} \text{Gini}(S_v)}$$

where S_v is the subset of examples where feature $A = v$, and $|S_v|/|S|$ is the fraction that goes into that branch.

Choosing the best split: Select the attribute A^* that *minimises* $\text{Gini}(A)$ (or equivalently maximises the Gini Gain $= \text{Gini}(S) - \text{Gini}(A)$).

Example

Gini Impurity for Outlook split:
Subsets (verify against dataset):

- Outlook = Rainy: ex{1,2,8,10} $\Rightarrow$ 1 Yes, 3 No. $\text{Gini}_{\text{Rainy}} = 1 - (1/4)^2 - (3/4)^2 = 1 - 0.0625 - 0.5625 = 0.375$
- Outlook = Overcast: ex{3,7,11} $\Rightarrow$ 3 Yes, 0 No. $\text{Gini}_{\text{Overcast}} = 1 - 1^2 - 0^2 = 0$ (pure!)
- Outlook = Sunny: ex{4,5,6,9,12} $\Rightarrow$ 3 Yes, 2 No. $\text{Gini}_{\text{Sunny}} = 1 - (3/5)^2 - (2/5)^2 = 1 - 0.36 - 0.16 = 0.48$

$$\text{Gini(Outlook)} = \frac{4}{12}(0.375) + \frac{3}{12}(0) + \frac{5}{12}(0.48) = 0.125 + 0 + 0.200 = \mathbf{0.325}$$

Gini Gain(Outlook) $= 0.486 - 0.325 = 0.161$.

Example

Gini Impurity for Humidity split:

- High: ex{1,2,3,4,8,11,12} $\Rightarrow$ 3 Yes, 4 No. $\text{Gini}_{\text{High}} = 1 - (3/7)^2 - (4/7)^2 = 1 - 9/49 - 16/49 = 24/49 \approx 0.490$
- Normal: ex{5,6,7,9,10} $\Rightarrow$ 4 Yes, 1 No. $\text{Gini}_{\text{Normal}} = 1 - (4/5)^2 - (1/5)^2 = 1 - 0.64 - 0.04 = 0.320$

$$\text{Gini(Humidity)} = \frac{7}{12}(0.490) + \frac{5}{12}(0.320) = 0.286 + 0.133 = \mathbf{0.419}$$

Gini Gain(Humidity) $= 0.486 - 0.419 = 0.067$.
Since $0.325 < 0.419$, **Outlook produces a lower Gini** — Outlook is the better split.

4 Entropy and Information Gain

4.1 Entropy of a Set

$$\boxed{H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k}$$

where the convention $0 \cdot \log_2 0 = 0$ is used for pure classes.

Properties:

- $H(S) = 0$: pure set.
- $H(S) = 1$ bit: maximally impure binary set (50/50 split).
- $H(S) \leq \log_2 K$ bits in general (K classes).
- Entropy is measured in **bits** when $\log_2$ is used.

Real-Life Analogy: Entropy measures *surprise* or *uncertainty*. If you know a coin is always heads ($p = 1$), there is no surprise — entropy $= 0$. If it is a fair coin, maximum surprise — entropy $= 1$ bit. High entropy in a dataset means the class labels are very mixed (unpredictable); low entropy means mostly one class (predictable).

4.2 Entropy After a Split

$$\boxed{H(S \mid A) = \sum_{\text{values } v \text{ of } A} \frac{|S_v|}{|S|} H(S_v)}$$

This is called the **conditional entropy** of S given attribute A.

4.3 Information Gain

$$\boxed{\text{IG}(S, A) = H(S) - H(S \mid A)}$$

Information Gain = entropy *before* split minus entropy *after* split.

- IG ≥ 0 always (splitting never increases average entropy).
- Higher IG $\Rightarrow$ better split (more information gained about class label after knowing A).
- Select the attribute A^* with the **highest** Information Gain.

Example

Step 1: Entropy of the full golf dataset:
$p_{\text{Yes}} = 7/12 \approx 0.5833$, $p_{\text{No}} = 5/12 \approx 0.4167$.

$$\begin{aligned} H(S) &= -(7/12)\log_2(7/12) - (5/12)\log_2(5/12) \\ &= -(0.5833)(-0.7779) - (0.4167)(-1.2630) \\ &= 0.4538 + 0.5263 = \mathbf{0.980} \text{ bits} \end{aligned}$$

Example

Step 2: Information Gain for Outlook:

- H_{Rainy}: 1 Yes, 3 No. $= -(1/4)\log_2(1/4) - (3/4)\log_2(3/4) = 0.500 + 0.311 = 0.811$ bits
- H_{Overcast}: 3 Yes, 0 No. $= 0$ bits (pure).
- H_{Sunny}: 3 Yes, 2 No. $= -(3/5)\log_2(3/5) - (2/5)\log_2(2/5) = 0.442 + 0.529 = 0.971$ bits

$$H(S \mid \text{Outlook}) = \frac{4}{12}(0.811) + \frac{3}{12}(0) + \frac{5}{12}(0.971) = 0.270 + 0 + 0.405 = 0.675 \text{ bits}$$

$$\text{IG}(S, \text{Outlook}) = 0.980 - 0.675 = \mathbf{0.305} \text{ bits}$$

Example

Step 3: Information Gain for all attributes:

Attribute	$H(S \mid A)$	IG(bits)	Rank
Outlook	0.675	**0.305**	**1st (Best)**
Humidity	0.875	0.105	2nd
Temperature	0.918	0.062	3rd
Windy	0.959	0.021	4th

Decision: Split on **Outlook** at the root node.
Derivation for Humidity: High: 3 Yes, 4 No $\Rightarrow H_{\text{High}} = -(3/7)\log_2(3/7) - (4/7)\log_2(4/7) = 0.524 + 0.461 = 0.985$ bits.
Normal: 4 Yes, 1 No $\Rightarrow H_{\text{Normal}} = -(4/5)\log_2(4/5) - (1/5)\log_2(1/5) = 0.258 + 0.464 = 0.722$ bits.
$H(S|\text{Humidity}) = (7/12)(0.985) + (5/12)(0.722) = 0.574 + 0.301 = 0.875$ bits.

4.4 Entropy vs. Gini — Which to Use?

Property	Entropy (IG)	Gini Impurity
Formula	$-\sum p_k \log_2 p_k$	$1 - \sum p_k^2$
Computation	Requires log; slower	No log; faster
Select split by	Maximise IG	Minimise Gini (or max Gini Gain)
Used in	ID3, C4.5	CART, scikit-learn
Bias	Slightly prefers balanced splits	Similar behaviour in practice

Key Idea

In practice, Gini impurity and entropy usually select the same attribute. The choice between them rarely makes a significant difference in prediction performance. Gini is faster to compute (no logarithm), so it is default in most libraries (e.g. scikit-learn's `DecisionTreeClassifier`).

5 Decision Tree Stopping Criteria

Without stopping rules, the algorithm builds a tree until every leaf is perfectly pure — but this causes severe overfitting. Stopping criteria define when to stop splitting and declare a leaf.

5.1 List of Standard Stopping Criteria

1. **All samples in the node belong to the same class.**
   The node is pure; no split is needed.
2. **No remaining attributes to split on.**
   All available features have been exhausted. Predict the majority class at the leaf.
3. **Impurity reduction (or Information Gain) below a threshold δ.**
   If the best available split reduces Gini/entropy by less than δ, stop. The gain is too small to justify more complexity.
4. **Maximum depth reached.**
   The tree has grown to a pre-defined maximum number of levels. Shallower trees are simpler and less likely to overfit.
5. **Minimum number of samples in a node to split (`min_samples_split`).**
   A node with too few examples should not be split further (not statistically reliable).
6. **Minimum number of samples per leaf (`min_samples_leaf`).**
   Each resulting child node must have at least a minimum number of examples.

Key Idea

Hyperparameter tuning for stopping criteria:
Max depth and min samples are hyperparameters — they are set *externally* and tuned using cross-validation.

- Small max depth $\Rightarrow$ simple tree, possibly underfitting.
- Large max depth $\Rightarrow$ complex tree, possibly overfitting.
- Tune with the validation fold to find the sweet spot.

5.2 Edge Cases and How to Handle Them

Q1: What if after exhausting all attributes, some examples in a leaf have different classes?

Answer: The dataset may be noisy or labels may depend on features not provided. Assign the leaf the **majority class** among examples in that node.

Q2: What if a split results in an empty child node (no examples in that branch)?

Answer: Assign the empty leaf the majority class of the *parent* node.

Q3: How do we split on continuous (numeric) attributes?

Answer: For a continuous feature x:

1. Sort the unique values of x in the training set.
2. Consider splitting at the midpoint between every pair of consecutive sorted values as candidate thresholds.
3. For each threshold t: compute IG (or Gini gain) for split $x \leq t$ vs $x > t$.
4. Choose the threshold t^* that maximises IG.
5. The split $x \leq t^*$ defines a binary partition.

Example

Continuous split example: Suppose Study Hours values in a dataset are: 1, 2, 3, 5, 6, 8.
Candidate thresholds: 1.5, 2.5, 3.5, 4.0, 5.5, 7.0.
For each threshold, compute IG and pick the best.
If threshold 3.5 perfectly separates Fail (≤ 3.5) from Pass (> 3.5), then IG = 1.0 bit (maximum) — a perfect split.

Q4: How are decision boundaries formed for continuous attributes?

Each binary split on a continuous feature creates an **axis-aligned threshold**: a vertical or horizontal line (in 2D feature space) at $x_j = t^*$. The resulting boundaries are step-function-shaped (boxes), not smooth curves.

6 The ID3 Algorithm

6.1 Overview

ID3 (Iterative Dichotomiser 3) was developed by Ross Quinlan in **1986**. It is the foundational decision tree learning algorithm.

Key Idea

ID3 in one sentence: Greedily build a decision tree by always splitting on the attribute with the highest *Information Gain* (entropy reduction).

6.2 ID3 Algorithm — Step by Step

1. If all examples in the current node have the same label, return a leaf node with that label.
2. If no attributes remain, return a leaf with the majority class.
3. For each available attribute A:
   (a) Compute $H(S)$ (entropy of current node).
   (b) Compute $H(S \mid A)$ (weighted entropy after split).
   (c) Compute $\mathrm{IG}(S, A) = H(S) - H(S \mid A)$.
4. Select $A^* = \arg\max_A \mathrm{IG}(S, A)$.
5. Split the dataset on A^*.
6. For each value v of A^*:
   (a) Create a child node for the subset S_v.
   (b) Remove A^* from available attributes.
   (c) Recursively apply ID3 to S_v.
7. Return the resulting subtree.

Example

Running ID3 on the Golf Dataset (root level):
From our calculations (Section 4.3):
IG(Outlook) = 0.305 bits ← winner.
Root node splits on Outlook:

- Branch **Overcast**: all 3 examples are Yes ⇒ **Leaf: Yes** (pure, depth 1).
- Branch **Rainy** (ex{1,2,8,10}): 1 Yes, 3 No. Recurse with remaining attributes {Temperature, Humidity, Windy}.
  Among these, split on Windy:
  - Windy=True (ex{2}): No ⇒ **Leaf: No**
  - Windy=False (ex{1,8,10}): No, No, Yes ⇒ majority class: No ⇒ **Leaf: No**
- Branch **Sunny** (ex{4,5,6,9,12}): 3 Yes, 2 No. Recurse.

Split on Humidity (highest IG in this subset):
- High (ex{4,12}): Yes, No ⇒ majority: Yes or further split needed.
- Normal (ex{5,9}): Yes, Yes ⇒ **Leaf: Yes** (pure).

6.3 Advantages of ID3

- Simple, elegant, and easy to implement.
- At each step picks the *most informative* attribute.
- Works well on small datasets with low-cardinality categorical attributes.
- Produces trees that are easy to interpret and visualise.

6.4 Limitations of ID3

- **Overfitting:** ID3 grows a full tree without pruning. On noise-containing data it memorises training examples.
- **No continuous attributes:** Cannot handle numerical features directly; requires discretisation preprocessing.
- **Bias toward high-cardinality attributes:** A feature with many unique values (e.g. a student ID) creates many small subsets, each looking pure — but this is meaningless and does not generalise.

Common Mistake / Pitfall

ID3 Bias Illustrated: Suppose we add a feature "Unique ID" (values 1–12, one per example). Every subset created by splitting on Unique ID has exactly 1 example ⇒ each leaf is pure ⇒ IG = $H(S) \approx 0.98$ bits. ID3 would choose Unique ID as the root split — but this is perfectly useless for prediction on new data. C4.5's Gain Ratio fixes this.

7 The C4.5 Algorithm

7.1 Overview

C4.5 was developed by Ross Quinlan in **1993** as a direct improvement over ID3. It addresses three major shortcomings of ID3:

1. Cannot handle continuous attributes.
2. No mechanism for missing values.
3. No pruning (prone to overfitting).
4. Biased towards high-cardinality attributes.

7.2 Gain Ratio (Replacing Information Gain)

C4.5 replaces Information Gain with **Gain Ratio** to penalise attributes that produce many small branches:

$$\boxed{\text{GainRatio}(S, A) = \frac{\text{IG}(S, A)}{\text{SplitInfo}(S, A)}}$$

where the **Split Information** penalises the attribute for the number (and balance) of branches it creates:

$$\boxed{\text{SplitInfo}(S, A) = -\sum_{v} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}}$$

Here v ranges over all values of A, and $|S_v|$ is the number of examples in subset v.

Intuition:

- SplitInfo is the entropy of the split itself (treating branches as events, independently of class labels).
- An attribute with many unique values has high SplitInfo.
- Dividing IG by SplitInfo *normalises* the gain — an attribute that creates many useless small branches is penalised.

Example

Gain Ratio for Outlook vs. a Unique-ID attribute:
Outlook (3 values: Rainy/Overcast/Sunny):

$$\text{SplitInfo(Outlook)} = -\frac{4}{12}\log_2\frac{4}{12} - \frac{3}{12}\log_2\frac{3}{12} - \frac{5}{12}\log_2\frac{5}{12} \approx 0.529 + 0.528 + 0.527 = 1.577 \text{ bits}$$

$$\text{GainRatio(Outlook)} = \frac{0.305}{1.577} \approx \mathbf{0.193}$$

Unique-ID attribute (12 values, each subset has 1 example):

$$\text{SplitInfo(UniqueID)} = -12 \times \frac{1}{12}\log_2\frac{1}{12} = \log_2 12 \approx 3.585 \text{ bits}$$

IG(UniqueID) $\approx$ 0.980 bits (entire entropy reduced to 0).

$$\text{GainRatio(UniqueID)} = \frac{0.980}{3.585} \approx 0.273$$

In this small dataset, UniqueID still wins on Gain Ratio. The Gain Ratio is most effective when the high-IG attribute has very many values (e.g., continuous IDs with thousands of distinct values), making SplitInfo extremely large and thereby suppressing it.

7.3 Handling Continuous Attributes

C4.5 handles numeric features using the same threshold approach described in Section 5:

- Sort the unique values of the feature.
- Evaluate each midpoint as a candidate threshold.
- The Gain Ratio is computed for each candidate split.
- The best threshold is selected.

Example

C4.5 on the Student Pass/Fail Dataset:

Student	Study Hours	Attendance (%)	Pass (1) / Fail (0)
A	1	50	0 (Fail)
B	5	80	1 (Pass)
C	6	85	1 (Pass)
D	2	40	0 (Fail)
E	3	60	0 (Fail)
F	8	90	1 (Pass)

Overall class distribution: 3 Pass, 3 Fail. $H(S) = -0.5\log_2 0.5 - 0.5\log_2 0.5 = 1.0$ bit.

Study Hours sorted: 1, 2, 3, 5, 6, 8. Check threshold = 3.5 (between 3 and 5):

- Study ≤ 3.5 (A, D, E): 0 Pass, 3 Fail $\Rightarrow$ pure: H = 0.
- Study > 3.5 (B, C, F): 3 Pass, 0 Fail $\Rightarrow$ pure: H = 0.

$H(S \mid \text{Study} \leq 3.5) = 0$; IG = 1.0 − 0 = 1.0 bit (perfect split).

Attendance threshold 70 (between 60 and 80):

- Att ≤ 70 (A, D, E): 3 Fail $\Rightarrow$ pure: H = 0.
- Att > 70 (B, C, F): 3 Pass $\Rightarrow$ pure: H = 0.

IG = 1.0 (also a perfect split).

Both features give IG = 1.0; Gain Ratio selects based on SplitInfo. The tree assigns: if Study Hours ≤ 3.5, predict Fail; otherwise predict Pass.

7.4 Handling Missing Values

C4.5 uses a **probabilistic** strategy rather than discarding examples with missing attributes:

1. **During Split Selection:** The fraction of available (non-missing) examples is multiplied into the IG calculation.

$$\text{IG}_{\text{adjusted}}(S, A) = \frac{|S_{\text{known}}|}{|S|} \times \text{IG}(S_{\text{known}}, A)$$

2. **During Tree Construction:** An example with a missing value for attribute A is distributed *fractionally* to each child branch, proportional to the fraction of examples with each value of A in the training set.

3. **During Prediction:** A test example with a missing attribute is sent down all branches simultaneously. The final prediction is a **weighted sum** of the class probabilities from each branch.

7.5 Pruning in C4.5

C4.5 applies **post-pruning** (also called reduced-error pruning or pessimistic pruning):

1. Grow the full tree.
2. Starting from the leaves, evaluate each internal subtree.
3. Replace a subtree with a leaf if doing so does not *significantly* increase error on a held-out set.
4. Repeat until no further pruning improves performance.

This removes branches that "accidentally" fit training noise, improving generalisation.

7.6 Advantages and Limitations of C4.5

Advantages:

- Handles both categorical *and* continuous attributes.
- Deals with missing values gracefully.
- Reduces overfitting through pruning.
- Gain Ratio mitigates the high-cardinality bias of ID3.
- More robust and widely applicable than ID3.

Limitations:

- Gain Ratio calculations require more computation.
- Can still overfit if not tuned carefully.
- Sometimes biased toward attributes with high information gain even after ratio correction.
- Not naturally suited for regression tasks (classification only).

8 Comparison: ID3 vs. C4.5

Property	ID3	C4.5
Year	1986	1993
Split criterion	Information Gain	Gain Ratio
Continuous attributes	**No**	**Yes** (threshold)
Missing values	**No**	**Yes** (probabilistic)
Pruning	**No**	**Yes** (post-pruning)
Task type	Classification only	Classification only
High-cardinality bias	Yes	Reduced
Computational cost	Low	Higher
Robustness	Moderate	High

Key Idea

C4.5 is a strictly better algorithm than ID3 in almost all scenarios. ID3 is studied primarily for its historical significance and pedagogical simplicity — the core idea (greedy IG maximisation) underpins all modern tree algorithms, including CART, Random Forests, and Gradient Boosted Trees.

9 The CART Algorithm

9.1 Overview

CART stands for **Classification And Regression Trees**. Developed by Breiman et al. (1984), CART is the most widely used decision tree algorithm and is the backbone of popular libraries like scikit-learn.

Key Idea

Key distinction of CART:

- CART is used for *both* classification *and* regression tasks.
- All splits are **binary** (exactly 2 child nodes).
- Different splitting criteria for each task type.

9.2 How CART Works

1. Start with the entire training dataset at the root.
2. **Select the best feature and threshold** to split:
   - *Classification:* minimise weighted Gini impurity.
   - *Regression:* minimise weighted MSE.

3. **Split the dataset** into exactly two child nodes ($\leq t^*$ and $> t^*$).
4. **Recursively repeat** on each child until a stopping criterion is met.
5. **Assign prediction at leaf nodes:**
   - *Classification:* majority class in leaf.
   - *Regression: mean* of target values in leaf.

9.3 Splitting Criteria

Task	Criterion	Goal
Classification	Gini Impurity	Minimise $\sum_v \frac{\lvert S_v \rvert}{\lvert S \rvert} \text{Gini}(S_v)$
Regression	Mean Squared Error (MSE)	Minimise $\sum_v \frac{\lvert S_v \rvert}{\lvert S \rvert} \text{MSE}(S_v)$

For regression, the MSE of a leaf is:

$$\text{MSE}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y})^2, \quad \bar{y} = \frac{1}{|S|} \sum_{i \in S} y_i$$

where $\bar{y}$ is the mean output value in the leaf.

9.4 CART Stopping Criteria

CART uses the same stopping criteria as general decision trees (Section 5):

- Maximum depth.
- Minimum samples per node to split (`min_samples_split`).
- Minimum samples per leaf (`min_samples_leaf`).
- Reduction in impurity (Gini or MSE) below a threshold.

9.5 CART for Regression — Worked Example

Example

Dataset: Predicting student exam marks from Study Hours and Attendance.

Student	Study Hrs (x_1)	Attendance (x_2)	Marks (y)
A	1	50	35
B	5	80	75
C	6	85	82
D	2	40	40
E	3	60	55
F	8	90	92

$\bar{y}_{\text{all}} = (35 + 75 + 82 + 40 + 55 + 92)/6 = 379/6 \approx 63.2.$
$\text{MSE}_{\text{all}} = \frac{1}{6} \sum (y_i - 63.2)^2 = \frac{1}{6}[(35-63.2)^2 + (75-63.2)^2 + (82-63.2)^2 + (40-63.2)^2 +$

$(55-63.2)^2 + (92-63.2)^2] = \frac{1}{6}[795.24 + 139.24 + 353.44 + 538.24 + 67.24 + 829.44]$
$= \frac{2723}{6} \approx \mathbf{453.8}$.

CART tries Study Hrs threshold at 3.5:

- Left (≤ 3.5): A(35), D(40), E(55). $\bar{y}_L = 130/3 \approx 43.3$. $\text{MSE}_L \approx (35-43.3)^2 + (40-43.3)^2 + (55-43.3)^2 = 69.1 + 10.9 + 137.1)/3 = 72.4$.
- Right (> 3.5): B(75), C(82), F(92). $\bar{y}_R = 249/3 = 83$. $\text{MSE}_R = (75-83)^2 + (82-83)^2 + (92-83)^2 = (64 + 1 + 81)/3 = 48.7$.

Weighted MSE $= (3/6)(72.4) + (3/6)(48.7) = 36.2 + 24.35 = \mathbf{60.55}$

This is a massive reduction from 453.8 — an excellent split.

Final tree after this split:

- If Study Hrs ≤ 3.5: predict $\hat{y} = 43.3$.
- If Study Hrs > 3.5: predict $\hat{y} = 83$.

10 Summary: ID3 vs. C4.5 vs. CART

Feature	ID3	C4.5	CART
Year	1986	1993	1984
Split metric	Info Gain	Gain Ratio	Gini / MSE
Split type	Multi-way	Multi-way	Binary only
Task	Classification	Classification	Class + Regression
Numeric features	No	Yes	Yes
Missing values	No	Yes	No (by default)
Pruning	No	Yes	Via hyperparams
Leaf output	Majority class	Majority class	Class or Mean
Used in	Education	Research	scikit-learn

11 Practice Questions

Practice Questions

Topic: Gini Impurity

Q1 (Easy): Compute the Gini impurity of a set with 10 examples: 8 of class A and 2 of class B.

Answer: $p_A = 0.8$, $p_B = 0.2$. Gini $= 1 - 0.64 - 0.04 = \mathbf{0.32}$.

Q2 (Medium): Two possible splits for a 20-example set (10 Yes/10 No):

- Split A: Left (8 Yes, 2 No, $n = 10$), Right (2 Yes, 8 No, $n = 10$).
- Split B: Left (5 Yes, 5 No, $n = 10$), Right (5 Yes, 5 No, $n = 10$).

Compute Gini(split A) and Gini(split B). Which is better?

Answer: $\text{Gini}_{L_A} = 1 - (0.8)^2 - (0.2)^2 = 0.32$; $\text{Gini}_{R_A} = 1 - (0.2)^2 - (0.8)^2 = 0.32$.
$\text{Gini}(A) = 0.5(0.32) + 0.5(0.32) = \mathbf{0.32}$.
$\text{Gini}_{L_B} = \text{Gini}_{R_B} = 1 - 0.25 - 0.25 = 0.50$. $\text{Gini}(B) = \mathbf{0.50}$.
Split A is better (lower Gini = less impurity after split). Split B creates 50/50 mixed nodes — completely useless.

Topic: Entropy / Information Gain

Q3 (Easy): Compute the entropy of a node with 6 Yes, 6 No (12 total).
Answer: $p = 0.5$ for both. $H = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1.0$ bit. (Maximum possible entropy for binary classification.)

Q4 (Exam-level): After splitting the 12-example golf dataset on Windy (False: 4 Yes / 2 No; True: 3 Yes / 3 No), compute IG(Windy).
Answer: $H(S) = 0.980$ bits (from Section 4.3).
$H_{\text{False}} = -(4/6)\log_2(4/6) - (2/6)\log_2(2/6) = 0.918$ bits.
$H_{\text{True}} = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = 1.0$ bit.
$H(S|\text{Windy}) = (6/12)(0.918) + (6/12)(1.0) = 0.459 + 0.500 = 0.959$ bits.
$\text{IG}(\text{Windy}) = 0.980 - 0.959 = \mathbf{0.021}$ bits. (Compare to IG(Outlook) = 0.305 — Outlook is much better.)

Topic: Gain Ratio and C4.5

Q5 (Medium): An attribute A has IG = 0.4 bits and creates 4 equally sized subsets ($|S_v|/|S| = 0.25$ for all v). Compute SplitInfo and Gain Ratio.
Answer: $\text{SplitInfo} = -4 \times 0.25 \log_2 0.25 = -4 \times 0.25 \times (-2) = 2.0$ bits.
$\text{GainRatio} = 0.4/2.0 = \mathbf{0.20}$.

Q6 (Medium): Another attribute B has IG = 0.35 bits and creates 2 equally sized subsets. Compute its Gain Ratio. Which attribute does C4.5 prefer, A or B?
Answer: $\text{SplitInfo}(B) = -2 \times 0.5 \log_2 0.5 = 1.0$ bit. $\text{GainRatio}(B) = 0.35/1.0 = 0.35$.
C4.5 prefers **B** (GainRatio $0.35 > 0.20$), even though A has higher raw IG. B's simpler two-way split is less likely to overfit.

Topic: CART Regression

Q7 (Medium): Using the student marks dataset from Section 8.4, verify the MSE reduction for the left leaf (≤ 3.5 Study Hours: marks 35, 40, 55) with mean 43.33.
Answer: $\text{MSE}_L = [(35 - 43.33)^2 + (40 - 43.33)^2 + (55 - 43.33)^2]/3 = [69.4 + 11.1 + 136.1]/3 = 216.6/3 = \mathbf{72.2}$
This leaf's mean prediction is 43.33 marks; any new student studying ≤ 3.5 hours would be predicted to score around 43 marks.

Q8 (Exam-level): Explain why a decision tree for classification predicts the *majority class* at a leaf, while a regression tree predicts the *mean* of outputs. What is the mathematical justification for the mean?
Answer: For classification, the optimal constant prediction (minimising 0-1 loss) for a set of mixed-class examples is the majority class.
For regression, the optimal constant prediction minimising MSE is the mean: $\hat{y} = \bar{y}$ minimises $\sum_{i \in \text{leaf}} (y_i - c)^2$ over all constants c. Taking the derivative: $\frac{d}{dc} \sum (y_i - c)^2 = -2 \sum (y_i - c) = 0 \Rightarrow c = \bar{y}$. The mean is the least-squares optimal constant — hence CART uses it at regression leaves.

12 Key Takeaways — Week 6

Quick Summary

- **Which attribute to split:** choose the one that maximises Information Gain (entropy) or Gini Gain (Gini).
- **Gini Impurity:** $\text{Gini}(S) = 1 - \sum p_k^2$. 0 = pure, 0.5 = maximally mixed (binary). Weighted Gini selects the split; minimise it.
- **Entropy:** $H(S) = -\sum p_k \log_2 p_k$. 0 = pure, 1 = maximally mixed (binary).
- **Information Gain:** $\text{IG}(S, A) = H(S) - H(S|A)$. Always ≥ 0; maximise it.
- **Golf dataset root split:** Outlook wins (IG = 0.305, Gini Gain = 0.161).
- **Stopping criteria:** pure node, no attributes left, IG/Gini below threshold, max depth, min samples per node/leaf.
- **Edge cases:** conflicting labels → majority class; empty subtree → parent majority; continuous → threshold search.
- **ID3** (1986): greedy IG maximisation; categorical only; no pruning; biased towards high-cardinality features.
- **C4.5** (1993): replaces IG with *Gain Ratio* (IG/SplitInfo); handles continuous attributes, missing values, and post-hoc pruning.
- **Gain Ratio**: $\text{GainRatio}(S, A) = \text{IG}(S, A)/\text{SplitInfo}(S, A)$; penalises attributes that create many small branches.
- **CART** (1984): binary splits only; Gini for classification, MSE for regression; used in scikit-learn.
- **CART leaf predictions:** majority class (classification) or mean target value (regression — minimises MSE).
- **Prefer**: ID3 for education; C4.5 for robust classification; CART for production code and ensemble methods.